

Per rendere i taccuini dei grafici (**tick**) professionali come quelli di MATLAB o gnuplot, non basta dividere l'asse in parti uguali con numeri decimali arbitrari (es. 0.137). I grafici professionali utilizzano l'algoritmo dei **"Nice Numbers"** (numeri tondi e leggibili come 1, 2, 5, 10, 20, 50, 0.1, 0.2, 0.5) unito a **tacchette visive (tick marks)** e una **griglia di sfondo leggera**.

L'algoritmo di Paul Heckbert (*Graphics Gems*) per calcolare intervalli eleganti e la sua integrazione nel rendering Win32 GDI si articolano come segue.

1. Funzione per calcolare i "Nice Numbers"

Aggiungi questa funzione helper nel file util.h per generare i valori dei tick basati sull'estensione dell'asse:

```
C++
#include <vector>
#include <cmath>

// Trova un numero "bello" vicino a x
inline double niceNum(double x, bool round) {
    int exp = static_cast<int>(std::floor(std::log10(x)));
    double f = x / std::pow(10.0, exp); // frazione tra 1 e 10
    double niceF;

    if (round) {
        if (f < 1.5) niceF = 1.0;
        else if (f < 3.0) niceF = 2.0;
        else if (f < 7.0) niceF = 5.0;
        else niceF = 10.0;
    } else {
        if (f <= 1.0) niceF = 1.0;
        else if (f <= 2.0) niceF = 2.0;
        else if (f <= 5.0) niceF = 5.0;
        else niceF = 10.0;
    }
    return niceF * std::pow(10.0, exp);
}

// Genera il vettore di posizioni per i tick
inline std::vector<double> generateNiceTicks(double minVal, double maxVal, int maxTicks = 6) {
    std::vector<double> ticks;
    if (minVal >= maxVal) return ticks;
```

```

double range = niceNum(maxVal - minVal, false);
double tickSpacing = niceNum(range / (maxTicks - 1), true);
double graphMin = std::floor(minVal / tickSpacing) * tickSpacing;
double graphMax = std::ceil(maxVal / tickSpacing) * tickSpacing;

for (double x = graphMin; x <= graphMax + (tickSpacing * 0.5); x += tickSpacing) {
    if (x >= minVal - 1e-9 && x <= maxVal + 1e-9) {
        ticks.push_back(x);
    }
}
return ticks;
}

```

2. Disegno dei Tick e della Griglia in WM_PAINT

In `plotter.cpp`, all'interno della gestione di `WM_PAINT`, crea una penna per la griglia e imposta l'allineamento del testo `Win32` (`SetTextAlign`) per centrare o allineare perfettamente i valori rispetto alle tacchette.

Sostituisci la vecchia scrittura dei numeri ai soli angoli con il seguente ciclo:

```

C++
// 1. Prepara le penne per il disegno
HPEN hGridPen = CreatePen(PS_DOT, 1, RGB(220, 220, 220)); // Griglia grigio chiara
tratteggiata
HPEN hAxisPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0)); // Assi e tick neri

// Salva il colore del testo e il modo di sfondo
SetBkMode(hDC, TRANSPARENT);
SetTextColor(hDC, RGB(0, 0, 0));

// --- TICKS & GRIGLIA ASSE X ---
std::vector<double> xTicks = generateNiceTicks(this->minx, this->maxx, 6);
SetTextAlign(hDC, TA_CENTER | TA_TOP); // Centra il testo sotto la tacchetta

for (size_t i = 0; i < xTicks.size(); ++i) {
    int px = Tx(xTicks[i]); // Converte in pixel schermo

    // A. Griglia verticale
    SelectObject(hDC, hGridPen);
    MoveToEx(hDC, px, this->yi, NULL);
    LineTo(hDC, px, this->yf);
}

```

```

// B. Tacchetta (Tick Mark) verso il basso sul bordo inferiore
SelectObject(hDC, hAxisPen);
MoveToEx(hDC, px, this->yf, NULL);
LineTo(hDC, px, this->yf + 5);

// C. Etichetta numerica
std::string label = formatNumber(xTicks[i]);
TextOut(hDC, px, this->yf + 7, label.c_str(), static_cast<int>(label.length()));
}

// --- TICKS & GRIGLIA ASSE Y ---
std::vector<double> yTicks = generateNiceTicks(this->miny, this->maxy, 6);
SetTextAlign(hDC, TA_RIGHT | TA_MIDDLE); // Allinea a destra rispetto alla tacchetta

for (size_t i = 0; i < yTicks.size(); ++i) {
    int py = Ty(yTicks[i]); // Converte in pixel schermo

    // A. Griglia orizzontale
    SelectObject(hDC, hGridPen);
    MoveToEx(hDC, this->xi, py, NULL);
    LineTo(hDC, this->xf, py);

    // B. Tacchetta (Tick Mark) verso sinistra sul bordo sinistro
    SelectObject(hDC, hAxisPen);
    MoveToEx(hDC, this->xi, py, NULL);
    LineTo(hDC, this->xi - 5, py);

    // C. Etichetta numerica
    std::string label = formatNumber(yTicks[i]);
    TextOut(hDC, this->xi - 8, py, label.c_str(), static_cast<int>(label.length()));
}

// Pulizia oggetti GDI creati
DeleteObject(hGridPen);
DeleteObject(hAxisPen);

```

Dettagli che rendono il grafico professionale

1. **Allineamento Win32 (SetTextAlign):** Senza TA_CENTER sul testo dell'asse X, i numeri "partono" dalla tacchetta invece di essere centrati sotto di essa. Per l'asse Y, TA_RIGHT | TA_MIDDLE garantisce che tutti i numeri (anche negativi o con più cifre) finiscano esattamente a sinistra del tick.

2. **Griglia sottile (PS_DOT):** La griglia aiuta la lettura senza appesantire la linea del grafico principale.
3. **Margine esterno dinamico:** Poiché le etichette dell'asse Y ora sporgono a sinistra di qualche pixel, assicurati che il valore del margine predefinito (margin = 40 o 50) lasci abbastanza spazio per evitare che i numeri vengano tagliati dal bordo della finestra.